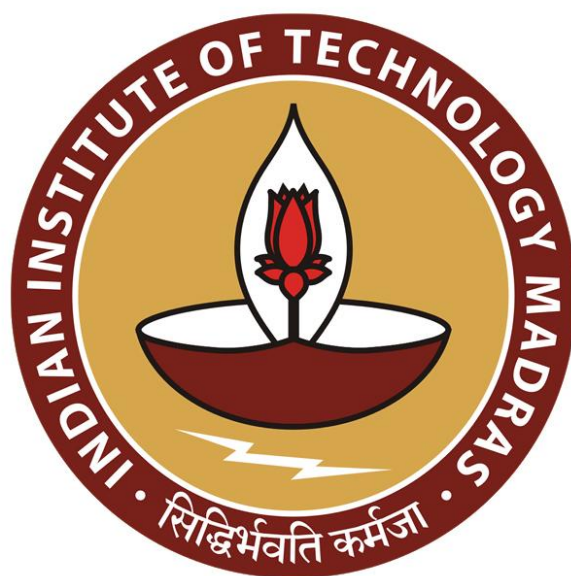




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Thread in Java

(Refer Slide Time: 00:13)



Threads in Java

Madhavan Mukund
<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java
Week 11



So last week, we looked briefly at how to create threads in Java. So, let us look at threads in a little more detail.

(Refer Slide Time: 00:19)



Creating threads in Java

- Have a class extend **Thread**
- Define a function **run()** where execution can begin in parallel
- Invoking **p[i].start()** initiates **p[i].run()** in a separate thread
 - Directly calling **p[i].run()** does not execute in separate thread!
- **sleep(t)** suspends thread for **t** milliseconds
 - Static function — use **Thread.sleep()** if current class does not extend **Thread**
 - Throws **InterruptedException** — later

```
public class Parallel extends Thread{
    private int id;
    public Parallel(int i){ id = i; }
    public void run(){
        for (int j = 0; j < 100; j++){
            System.out.println("My id is "+id);
            try{
                sleep(1000); // Sleep for 1000 ms
            } catch(InterruptedException e){}
        }
    }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            p[i].start(); // Start p[i].run()
                        // in concurrent thread
        }
    }
}
```





- Have a class extend `Thread`
- Define a function `run()` where execution can begin in parallel
- Invoking `p[i].start()` initiates `p[i].run()` in a separate thread
 - Directly calling `p[i].run()` does not execute in separate thread!
- `sleep(t)` suspends thread for `t` milliseconds
- Static function — use `Thread.sleep()` if current class does not extend `Thread`
- Throws `InterruptedException` — later

Typical output

```
My id is 0  
My id is 3  
My id is 2  
My id is 1  
My id is 4  
My id is 0  
My id is 2  
My id is 3  
My id is 4  
My id is 1  
My id is 5  
My id is 3  
My id is 1  
My id is 2  
My id is 4  
My id is 0  
...
```



Madhavan Mukund

Threads in Java

Programming Concepts using Java 2/7

So, remember that in Java, we can create threads that can run in parallel by extending the class `Thread`. And the way we do it is we extend the class `Thread`. And inside this, we have a special function called `run`. So, when we have something that extends `Thread` and override this function `run`, then that function `run` can be actually executed in parallel.

And the tricky thing is that to execute this function `run` in parallel, what you do is you call that function through `start`. So, here is how you do it. So, you have this class which extends `Thread`. And in this particular case, what it does is it prints out, so it has an only one instance variable, which is its `id`, and it just prints that out 100 times, and in between it sleeps for 1 second, 1000 milliseconds.

So, this is what each object of type `parallel` does. So, now this class at the bottom, this main function, it creates five such objects like it has an array of five such things. So, it creates, in `pi`, it creates a new `parallel` object whose `id` is `i`, and then it starts it. So, the important thing is that it starts and not `run`, I can of course, call `pi dot run`, but if I do `pi dot run`, it will not happen in parallel, it will be like a normal function call where it will wait for `pi dot run` to execute, and then come back.

So, if I do `pi dot run`, then the 0th process will print its `id` 100 times, then process one will print it 100 times and so on sequentially, whereas if I do `pi dot start`, then I get this kind of an interleaved behaviour, where in some sequence, the 0, 1, 2, 3, 4, these five processes keep printing their `id` and the order in which they printed keeps changing, because its the scheduling is kind of random.

(Refer Slide Time: 02:05)



Java threads ...

- Cannot always extend `Thread`
 - Single inheritance
- Instead, implement `Runnable`
- To use `Runnable` class, explicitly create a `Thread` and `start()` it

```
public class Parallel implements Runnable{
    // only the line above has changed
    private int id;
    public Parallel(int i){ ... } // Constructor
    public void run(){ ... }
}

public class TestParallel {
    public static void main(String[] args){
        Parallel p[] = new Parallel[5];
        Thread t[] = new Thread[5];

        for (int i = 0; i < 5; i++){
            p[i] = new Parallel(i);
            t[i] = new Thread(p[i]);
            // Make a thread t[i] from p[i]
            t[i].start(); // Start off p[i].run()
            // Note: t[i].start(),
            // not p[i].start()
        }
    }
}
```



Madhavan Mukund

Threads in Java

Programming Concepts using Java 3/7

So, we also said that because of this multiple inheritance problem, or single inheritance problem, it is not enough to just have one class thread that you can extend. So, we have to allow something more flexible. So, Java has a runnable interface, which has the same behaviour, you define your class by implementing runnable to extending thread and everything else the same, you have to override this function run.

But the difference is that when you are using a runnable class, rather than a thread extended subclass, you have to create the runnable objects, and then you have to make threads out of them. So, each `pi` has to be converted into a thread `ti`, and you need to start the thread, you cannot start the runnable object directly. So, it is against `start` and not `run`. But it is `start` with respect to the thread, which has been created with the runnable object. So, this is what we saw earlier.

(Refer Slide Time: 03:02)



IIT Madras
BSc Degree

Life cycle of a Java thread

A thread can be in six states — thread status via `t.getState()`

- **New:** Created but not `start()`ed.
- **Runnable:** `start()`ed and ready to be scheduled.
 - Need not be actually “running”
 - No guarantee made about how scheduling is done
 - Most Java implementations use time-slicing
- **Not available to run**
 - **Blocked** — waiting for a lock, unblocked when lock is granted
 - **Waiting** — suspended by `wait()`, unblocked by `notify()` or `notifyAll()`
 - **Timed wait** — within `sleep(..)`, released when sleep timer expires
- **Dead:** thread terminates.



Madhavan Mukund

Threads in Java

Programming Concepts using Java

4 / 7

So, a Java thread goes through a natural lifetime, like a Java function would, like you call the function it executes, it terminates. But in a Java thread, because it is occurring in parallel with other things, it is a little bit more complicated the type of things that a Java thread could do. So, initially, you have just created a thread, but you have not started it running. This Java thread is set to be in a new state.

Now, once you started, then it is said to be in a runnable state. Now, runnable does not mean running, running means it is currently executing. But remember that if you have say, for example, only a single CPU, you could have many things potentially which are active and therefore they are runnable. But only one of them can be running at a time. Moreover, Java does not promise you also in what order these things be scheduled.

So, in general, you will be doing it by allotting a certain amount of time per thread, then swapping it out putting another one. So, if you have even one resource only to compute on, you can have many threads running by each one for a small amount of time. But this is typically the case for you know, desktops and big computers. But this may not be the case for instance, if you are running Java on a phone.

Java on a phone might actually execute a thread until it finishes before it executes another thread. So, it really does depend on which Java implementation which platform you are running it on. So, there is no guarantee about how scheduling is done. So, runnable is a situation where modular the scheduler allowing it to run the thread can run.

But of course, there are situations when a thread may not be able to run. So, one situation is it is trying to execute something which is synchronized, it needs a lock but that lock is already taken by another thread. So, then it gets blocked. And it must wait for that thread to release it. And then the scheduler will decide whether to wake up this thread or something else that is blocked.

So, remember that, in this blocked state, the thread itself is not doing any checking, it is not a busy wait, it is just sleeping, it is the system, which is keeping track of which threads are blocked on which locks. And when the lock is released, it is a system, the underlying system, the operating system and the JVM job to go and find the threads which are waiting on this lock and wake them up and then decide which of them will run.

So, that is, if you are waiting for a synchronized lock. On the other hand, a thread can suspend itself, if you remember by saying wait, this is a condition variable wait. So, then you are also blocked when in different way. So, now you are not waiting for a lock to be released. But for some other thread to signal that this condition has now changed. So, there is some update in the state, which is through a notify, or notify all.

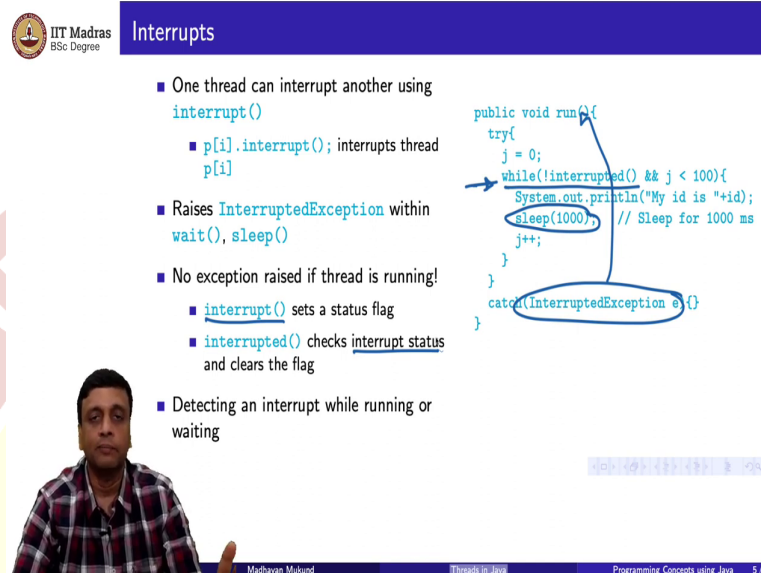
So, you could be blocked, or technically you can be waiting. So, Java distinguishes these two states, even though it does not distinguish the various kinds of runnable, running but not scheduled, you cannot check. But you can actually distinguish between blocked for a synchronization and waiting for a notify. And finally, we saw this sleep. We have sleep, we pass an argument which is interpreted as a length of time in milliseconds.

So, this is it also kind of wait, the thread is suspended, it is not doing anything. But here, the suspension lasts for a fixed amount of time. It is not based on some other event happening, which it cannot predict. But it knows that after a certain amount of time, there will be a signal saying come and wake up. And finally, like any other function, eventually, hopefully the thread will finish.

And then it goes into a dead state. So, we have six states, we have new, runnable, then these three versions of being blocked, and one dead state. And there is actually a function in Java called get state which you can apply to a thread and find out which state it is in. So, it will give you what kind of enumerated variable which will be one of these six things, not, wording is not exactly this, but it will give you one of these six things.

So, this is the life cycle of a Java thread, it begins as a new thread, then it becomes runnable. While it is runnable, periodically, it may block or it might wait or it might be in a time wait, and eventually it should end and it becomes dead.

(Refer Slide Time: 07:16)



IIT Madras
BSc Degree

Interrupts

- One thread can interrupt another using `interrupt()`
 - `p[i].interrupt();` interrupts thread `p[i]`
- Raises `InterruptedException` within `wait()`, `sleep()`
- No exception raised if thread is running!
 - `interrupt()` sets a status flag
 - `interrupted()` checks `interrupt` status and clears the flag
- Detecting an interrupt while running or waiting

```
public void run(){
    try{
        j = 0;
        while(!interrupted() && j < 100){
            System.out.println("My id is "+id);
            sleep(1000); // Sleep for 1000 ms
            j++;
        }
        catch(InterruptedException e){
        }
    }
}
```

Madhavan Mukund | Threads in Java | Programming Concepts using Java | 5/7

So, you might want to send a signal to another thread that something interesting has happened. Or you might just want to tell another thread that it should stop running. So, an interrupt is like what you would expect in the world, you go and tap somebody on the shoulder and say, excuse me. So, they are busy watching something, and you can tap them on the shoulder and then we interrupt them.

So, if I say give a thread and I say interrupt, then it sends a signal to the other thread that it is being interrupted. Now, the signal does not carry any information. So, it is up to the other thread to decide what to do with the signal. But the signal itself, just is a signal, it is just somebody tapping on your shoulder, you do not know who it is and why they are tapping. So, if you are sitting inside that thread, how do you know this?

Well, if the thread is inert, that is it is waiting, or it is sleeping, or even if it is blocked, then this signal will go as an exception. So, you will get an interrupted exception. So, that is the reason when we do a wait or a sleep, we have to do it in a try block checking this interrupted exception. But paradoxically, if it is running, if it is actually in a runnable state, then there is no interrupt exception, then you actually have to check.

And this is some kind of a flag, some kind of signal flag, it is like, somebody comes and pops up something on your object saying there is an interrupt. And now you have to go and look and see whether that flag is raised or not. So, there is a function. So, the interrupt function sets the flag. And inside the thread, you can say interrupted to check the flag. So, here is something that you might do if you want to catch both.

So, if you want to try, you are waiting for an interrupt and you want to do something, then either you run and every time before the loop, you check whether you have been interrupted or not. And if it is potentially the case that in between you are inactive, then you would also get an interrupt through the exception. So, one way or the other, you will catch it. If you only do the interrupted exception, you would not see this flag which is running.

And if you only do this running thing, then you would not catch the interrupted exception and because it is an exception is thrown by sleep if you do not catch this, then you must kind of advertise this outside your class saying that I throw interrupted exceptions, but in general you have to do both. So, you have to be able to check for interruption when you are running by calling this function interrupted.

And notice that what interrupted does is it checks the status and if the interrupt is there, if it is not there, of course it will say false. If it is true then it will reset the interrupt. So, this means that the interrupt will have to be regenerated the next time. So, you will not see the same interrupt multiple times. So, that is one nice thing about is interrupted thing. It actually, as you would expect, it clears this interrupt flag.

So, that now somebody else can interrupt you later. Now, as I mentioned, there is no guarantee as to what you should do with this interrupt. So, you have to interpret and maybe you have to check something to understand why you have been interrupted. So, there is no information conveyed with interrupt except that somebody is interpreting, there is a signal.

(Refer Slide Time: 10:30)



More about threads ...

- Check a thread's interrupt status
 - Use `t.isInterrupted()` to check status of `t`'s interrupt flag
 - Does **not** clear flag
- Can give up running status
 - `yield()` gives up active state to another thread
 - Static method in `Thread`
 - Normally, scheduling of threads is handled by OS — preemptive
 - Some mobile platforms use **cooperative scheduling** — thread loses control only if it yields
- Waiting for other threads
 - `t.join()` waits for `t` to terminate



Madhavan Mukund

Threads in Java

Programming Concepts using Java

6 / 7

So, you can also check this interrupted flag without clearing it, just find out whether something and not just your own flag, you can check for any thread, you can check the interrupt flag by using a different function called `isInterrupted`. So, `isInterrupted` is different from the interrupted thing. `Interrupted`, checks whether it is interrupted and then clears the flag. `isInterrupted` checks whether there is a pending interrupt somebody has set the flag, but does not clear the flag.

So, this is a, you want to think about a non-destructive examination of the interpretive status of another thing. Now, remember that we said that we there is no guarantee in general about how threads are scheduled. So, in a normal system, like in a desktop or a laptop, a thread will run for some time, and then automatically some other thread will run. So, the system will kind of guarantee that the resources are scheduled in such a way that no thread dominates.

But it may be the case that you want a thread to give up its running status. So, it is like you wanted to resign and say I will come back later. So, there is something called `yield`, which gives up the active status to another thread. So, basically, it is still runnable, it is not blocked. It is just saying, give some time to somebody else to run, I will come back and run later. So, this is a static method in thread.

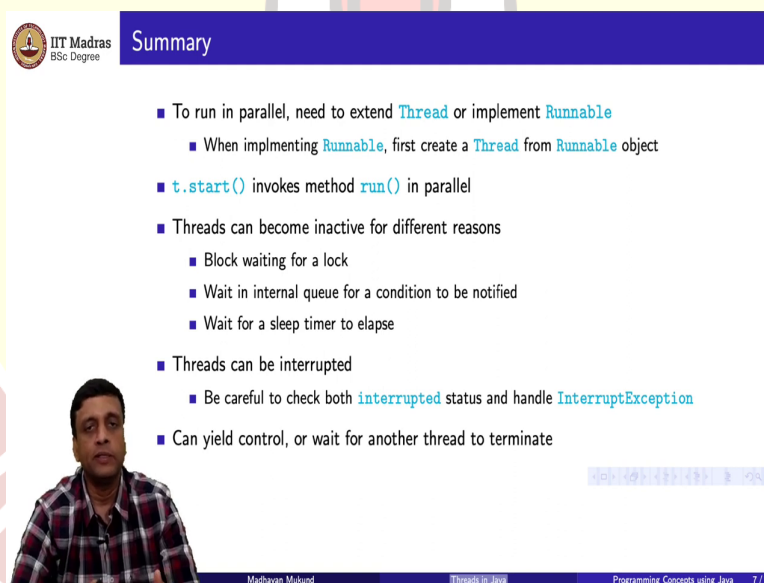
And this is used typically, you do not use it very often in normal Java programming, because the OS will actually guarantee that every thread yields periodically, it is no, no thread hogs the entire resources. But when you are doing things on a mobile environment, it may be that

the OS does not do, it allows a thread to run unless it yield, so depending it really depends on the kind of application you are writing in Java, whether you need to yield or not.

The other thing that you might want to do, so here, as we said, you can investigate whether another thread has been interrupted, so you get some information about the other thread. But it is quite common when things start in parallel. So, you start something and then you want to do some two or three things in parallel. So, you start some threads. And now each of these might want to wait for all the other threads to come and synchronize again.

So, you might want to wait for all the threads that were started in parallel to finish before proceeding. So, this you can do using a join, so you can wait for a thread to finish by saying `t.join()`. So, this will block the current thread until the thread `t` actually reaches the end of its execution reaches a dead state. So, `t` has to terminate before this. So, join is a blocking thing, which allows `t` to this thread to join with `t` in the sense that it waits for `t` to terminate.

(Refer Slide Time: 13:08)



Summary

- To run in parallel, need to extend `Thread` or implement `Runnable`
 - When implementing `Runnable`, first create a `Thread` from `Runnable` object
- `t.start()` invokes method `run()` in parallel
- Threads can become inactive for different reasons
 - Block waiting for a lock
 - Wait in internal queue for a condition to be notified
 - Wait for a sleep timer to elapse
- Threads can be interrupted
 - Be careful to check both `interrupted` status and handle `InterruptedException`
- Can yield control, or wait for another thread to terminate

Madhavan Mukund | Threads in Java | Programming Concepts using Java 7/7

So, to summarize, as we saw earlier, if you want to run something in parallel in Java, you must extend thread or implement runnable. And if you implement runnable, you have to be careful to first take the runnable object and make a thread out of it. Otherwise, you cannot do the start. And start invokes run in parallel. So, you do not directly call the run function you have to implement the run function override it.

But what you do to create run in parallel is to actually say start. So, thread once it starts it is runnable. So, when you create it, it is new when it starts is runnable. But every runnable

thread might go through a situation where it cannot run because it is blocked for various reasons, might be waiting for a lock, it had might have suspended itself and it is waiting for a notification, or it might be waiting for some time to elapse through a sleep. And the other thing you can do is you can send a signal to another thread.

So, a thread must be able to receive the signal, so it can check its interrupted status or it can catch an interrupted exception. And finally, you can yield control by saying I want to stop running temporarily and let some other thread takeover and you can also join with another thread which means you can specify that you want to wait until another thread is completed before you proceed.

So, this is a block like you are saying that now I want all my concurrent threads to again synchronize and proceed only after everything that is pending is finished.

